Ragnarok Research Proposal: A Sovereign, Always-On Personal AI Agent: Architecting Privacy-Preserving Autonomous Workflows with Persistent Memory on a Fully Local, Ollama-Based Stack

1. Context and Background

The proliferation of Large Language Model (LLM) based assistants has revolutionized human-computer interaction. However, current commercial AI assistants operate on a centralized, cloud-dependent paradigm, a distant Asgard whose halls the user may never enter. This architecture inherently compromises user privacy, as personal data, conversational contexts, and workflow habits are continuously harvested by third-party providers, carried off like tribute to a foreign hoard.

Furthermore, existing assistants are predominantly stateless or rely on limited context windows, rendering them incapable of true long-term personalization. Like a skald who forgets the saga by morning, they function as reactive chatbots rather than proactive, autonomous agents capable of executing complex, multi-step tasks across a user's diverse daily workflows.

Today, small open models can run on an ordinary laptop through Ollama, which serves chat, embeddings, and tool calling over a local address. The forge can now be raised inside the user's own hall, with nothing leaving the machine.

2. Problem Statement

Despite the rapid advancement of AI capabilities, there exists a critical dichotomy between agent autonomy and user data sovereignty. Current AI assistants fail to provide a unified, always-on operational framework that respects user privacy while delivering deep, context-aware workflow automation.

Specifically, the problem is threefold:

Loss of Data Sovereignty: Users do not own or control the data used to personalize their assistants, leading to privacy vulnerabilities and a lack of trust, their hoard plundered by distant jarls.

Contextual Amnesia: Existing systems lack robust, persistent memory architectures, like Mimir's well run dry. They cannot reliably retain long-term user preferences, historical context, or evolving project states, forcing users to repeatedly re-establish context.

Siloed and Ephemeral Execution: Current agents are restricted to isolated, single-turn interactions, each of the Nine Realms severed from the rest. They lack a modular, reusable skill framework and the orchestration capabilities required to autonomously navigate and execute tasks across heterogeneous, real-world daily workflows (e.g., transitioning from email triage to calendar scheduling to code generation).

Research Objective:

To design, build, and evaluate a localized, privacy-preserving, "always-on" personal AI agent, a hearth-bound guardian standing watch like Heimdall, where **every model call goes to a local Ollama instance**. The system must keep all data on the user's machine, remember past interactions, use reusable skills, and carry out multi-step tasks on its own, with no cloud AI service and no API key anywhere.

3. System Requirements

To address the aforementioned problem, the proposed system must satisfy the following functional and non-functional requirements, laws carved in runestone.

3.1 Functional Requirements (FR)

Functional requirements define the specific behaviors, capabilities, and functions the system must perform.

FR1: Persistent Memory Management

FR1.1 (Episodic Memory): The system must maintain a chronological log of user interactions, completed tasks, and contextual events, a saga-log kept by Muninn, allowing it to recall specific past occurrences.

FR1.2 (Semantic Memory): The system must extract, store, and retrieve generalized knowledge, user preferences, and factual data in a structured format (e.g., vector embeddings or knowledge graphs), the wisdom drawn from Mimir's well.

FR1.3 (Memory Consolidation & Forgetting): The system must autonomously consolidate short-term working memory into long-term storage and implement a "forgetting" mechanism to prune obsolete or irrelevant data to optimize retrieval, as the Norns weave what endures and cut what fades.

FR2: Modular Skill and Tool Execution

FR2.1 (Skill Registration): The system must allow users to define, register, and manage reusable "skills" (e.g., Python scripts, API wrappers, local CLI commands) via a standardized interface, each skill a rune inscribed in the dwarves' forge.

FR2.2 (Dynamic Tool Selection): Given a user prompt, the system must analyze the intent, select the appropriate combination of registered skills, and generate the necessary parameters for execution, choosing the right weapon for the battle.

FR2.3 (Sandboxed Execution): The system must execute these skills in isolated, sandboxed environments, behind a shield-wall, to prevent malicious or erroneous code from compromising the host system.

FR3: Autonomous Workflow Orchestration

FR3.1 (Multi-step Planning): The system must decompose high-level, ambiguous user goals into a sequence of actionable, logical sub-tasks (an execution graph), charting the voyage across the Nine Realms.

FR3.2 (Cross-Application Integration): The system must seamlessly interact with external, user-approved applications (e.g., email clients, IDEs, project management tools), crossing a Bifrost bridge, to carry out its planned sub-tasks.

FR3.3 (Self-Correction and Reflection): The system must monitor the output of its executed skills, evaluate if the output meets the expected criteria, and autonomously retry or adjust its plan upon failure, as Odin sought wisdom again and again at the well.

FR4: Proactive and Continuous Interaction

FR4.1 (Always-On Background Processing): The system must continuously monitor user-defined triggers (e.g., incoming emails, file changes, time-based cron jobs), a sleepless sentinel like Heimdall, to initiate tasks without explicit user prompting.

FR4.2 (Contextual Proactivity): Based on persistent memory and current context, the system must proactively suggest actions or provide relevant information to the user, as Odin's ravens return each dawn

with news.

3.2 Non-Functional Requirements (NFR)

Non-functional requirements define the system's operational constraints, quality attributes, and architectural boundaries.

NFR1: Privacy and Data Sovereignty (Security)

NFR1.1 (Local-First Architecture): All raw user data, memory stores, and execution logs must be stored locally on the user's hardware, kept within the walls of the user's own hall. Zero raw data shall be transmitted to external third-party servers.

NFR1.2 (Encrypted Memory at Rest): The persistent memory database must be encrypted at rest using user-held cryptographic keys, its runes sealed and only the user holding the key.

NFR1.3 (Fully Local LLM Inference): All reasoning and embedding calls must be served by local Ollama models, and no external LLM API may be used. Since no prompt leaves the machine, no PII-redaction pipeline is needed.

NFR2: Performance and Resource Efficiency

NFR2.1 (Responsiveness): The system must process and respond to direct user queries with a latency of less than 800ms for memory retrieval and intent classification, as swift as Sleipnir.

NFR2.2 (Resource Footprint): The "always-on" background monitoring and local model inference must not consume more than 30% of the host machine's baseline CPU/GPU resources, ensuring it does not impede the user's primary workflows.

NFR3: Reliability and Fault Tolerance

NFR3.1 (Graceful Degradation): If a specific external tool or API fails, the system must gracefully halt that specific sub-task, notify the user, and continue with independent sub-tasks rather than crashing the entire workflow, a single bough of Yggdrasil breaking while the tree still stands.

NFR3.2 (State Recovery): In the event of a system crash or power loss, the agent must be able to restore its exact operational state, including incomplete task graphs and working memory, within 5 seconds of reboot, rising again from Helheim.

NFR4: Usability and Transparency

NFR4.1 (Explainability): The system must provide a transparent "chain of thought" or execution log, a seer's telling, allowing the user to understand why a specific tool was chosen and how a decision was made.

NFR4.2 (User Control over Memory): The system must provide a user-friendly interface to view, edit, export, and permanently delete specific memories, ensuring the user retains ultimate control over the agent's knowledge base, as a jarl commands his own hall.

NFR4.3 (Permission Granularity): The system must enforce strict, granular Role-Based Access Control (RBAC), requiring explicit, just-in-time user approval before accessing sensitive tools or executing high-risk skills (e.g., sending an email, deleting a file), no gate opened until the gatekeeper's horn is sounded.

4. Conclusion

The realization of this system requires a paradigm shift from centralized, stateless AI chatbots to decentralized, stateful, and sovereign agents, a new world rising green from the sea after Ragnarok. By running every model, memory store, and skill on the user's own machine through Ollama, and by strictly adhering to the functional and non-functional requirements outlined above, this research will establish a foundational architecture for an always-on personal assistant that amplifies human productivity through persistent memory and autonomous workflow execution, without sacrificing the fundamental right to data privacy and user control.